Contents

1 Support Vector Machines (SVM)

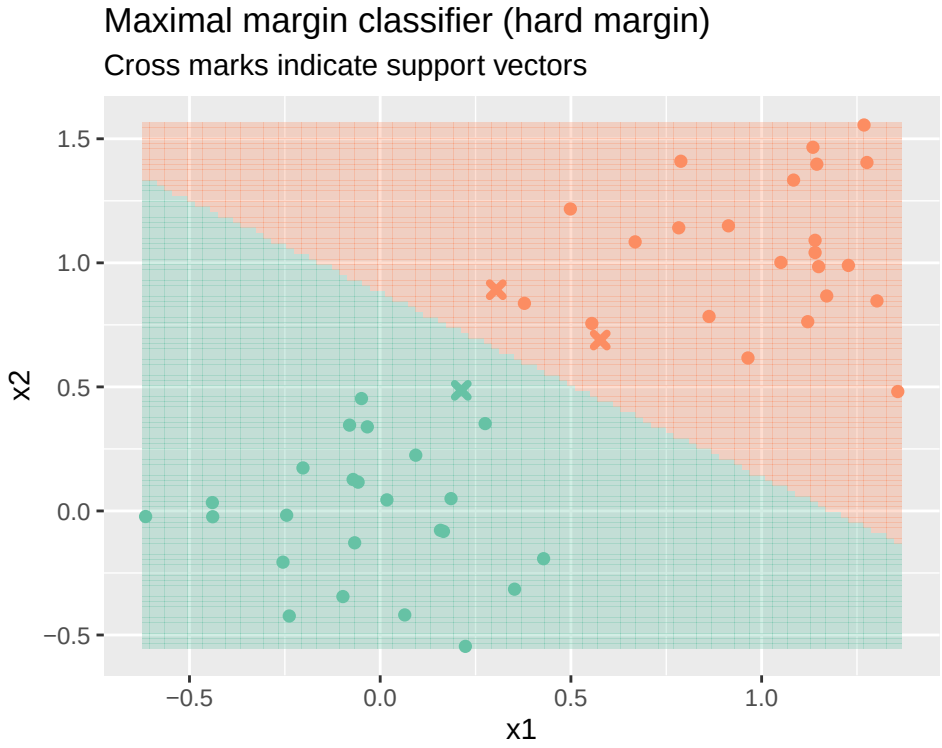
Broadly speaking, the support vector machine involves defining a *hyperplane* that separates the classes. A hyperplane is a surface that is 1 less dimensional than the data. For example, if our data is 2-dimensional, the hyperplane is 1-dimensional (a line). If our data is 27-dimensional, the hyperplane is 26-dimensional. For visualisation purposes in this tutorial, we will use mostly 2D/3D datasets, and so the hyperplane is 1D/2D.

The mathematics involved in defining this hyperplane is involved, and includes non-linear optimisation, which we will conveniently avoid. I will just focus on some visual intuition instead.

2 Maximal margin classifier (hard margin)

The first iteration of SVM we will see is a maximal margin classifier, which is when our classes are (linearly) *separable*, i.e. there is no overlap in classes. You can draw infinitely many lines that separate these two classes exactly, but the goal of the SVM is to maximise the **margin** between the two classes. The margin is defined as the distance between the separating hyperplane and the points closest to it, which are called **support vectors**.

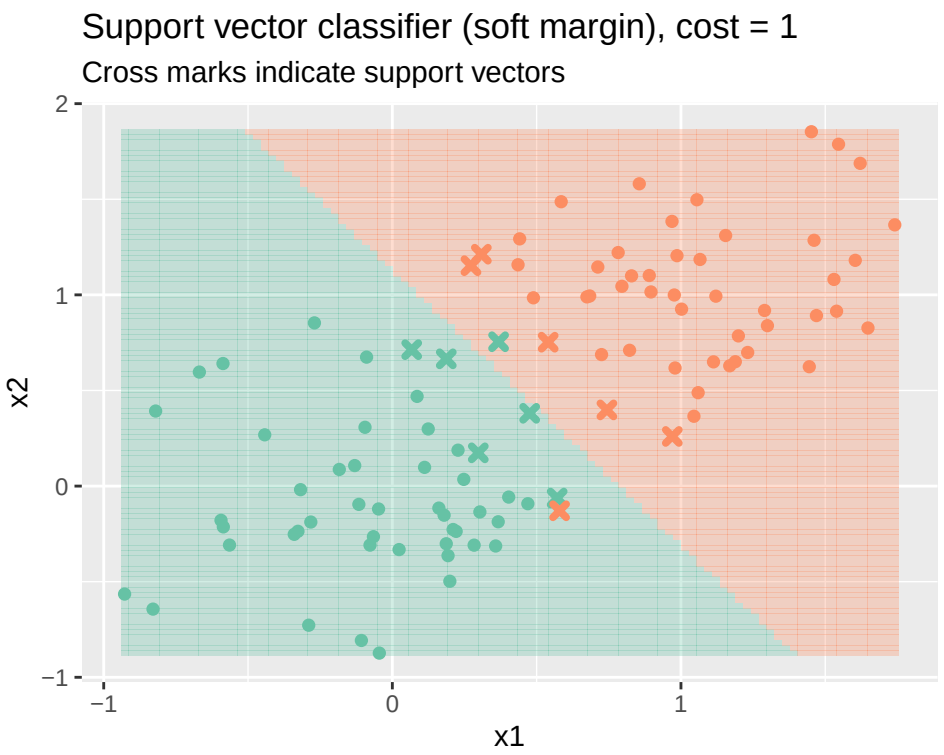
Every other point that is *not a support vector* had no influence on how this separating hyperplane was defined.

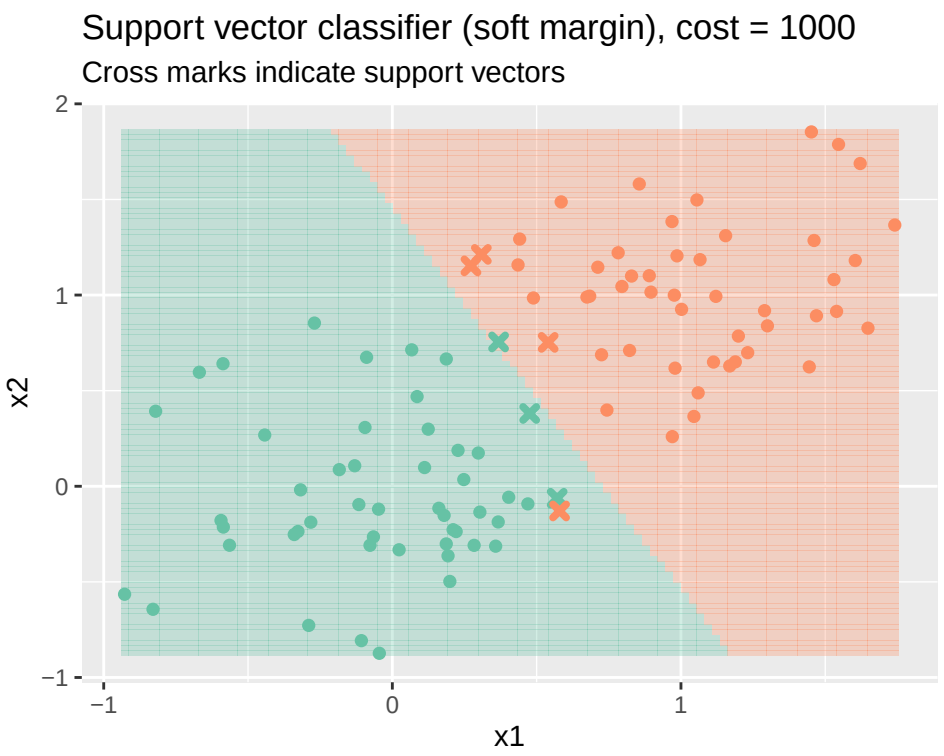


3 Support vector classifiers (soft margin)

When our data is not separable, (there is an overlap in classes), we cannot draw a hyperplane that will separate our two classes cleanly. Instead, every misclassified observation and observation lying within the margin is penalised at a **cost**. In the example below, minimally, one orange dot *must* be misclassified. The cost determines how much we penalise misclassfications, which determines how we will draw the hyperplane.

You may extend this to datasets which are even less separable, but the difference between models is most obvious when we have only a few misclassifications.



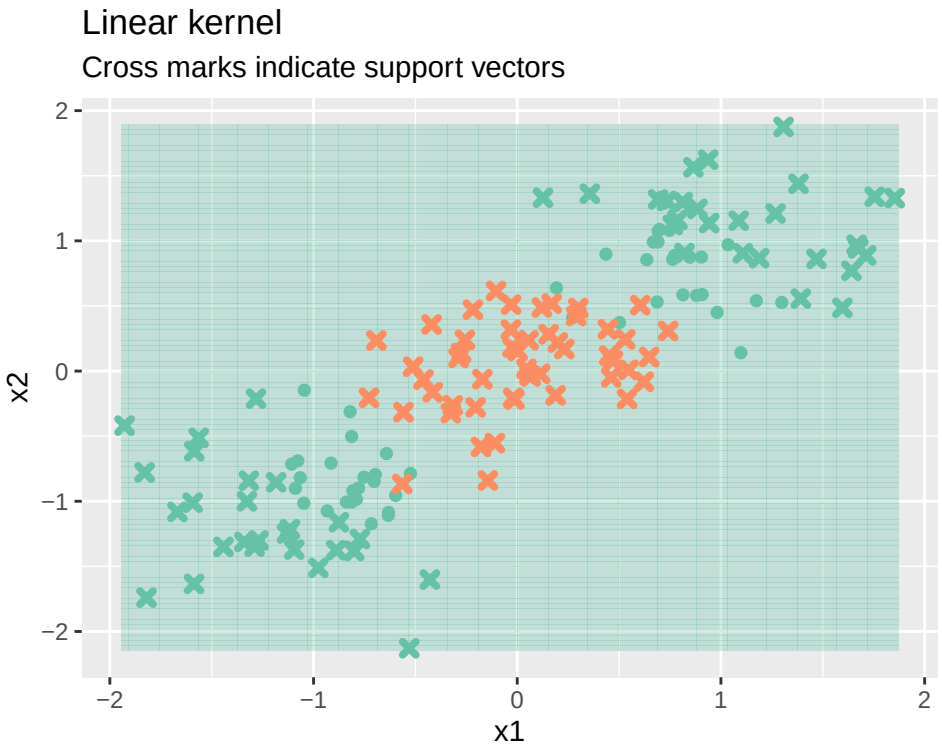


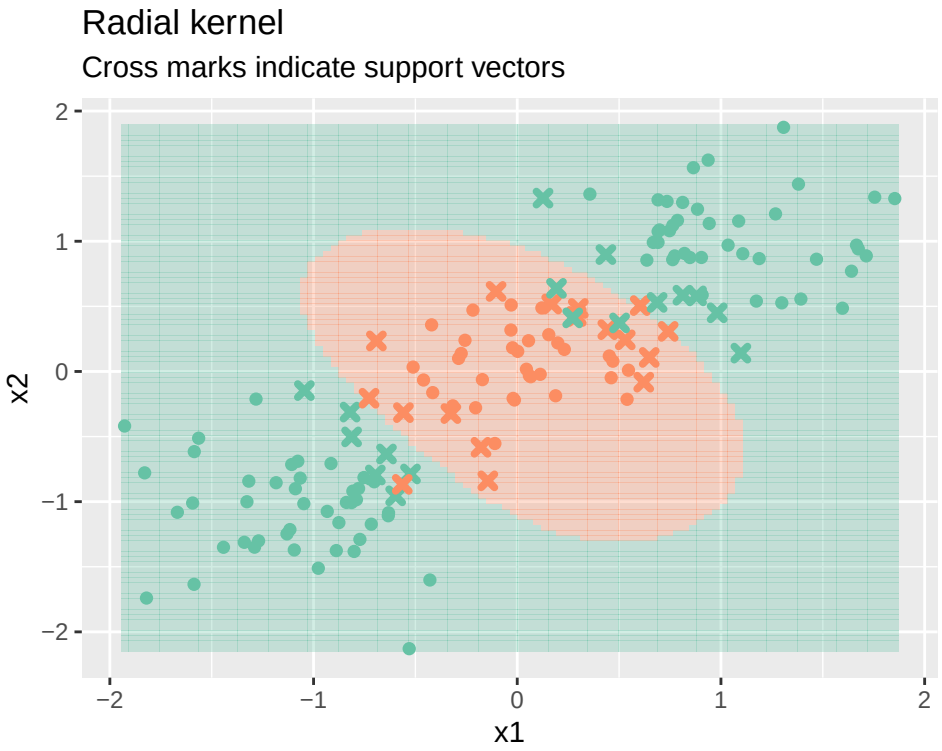
4 Nonlinear decision boundaries

More commonly, our data will not be linearly separable. There is a clear decision boundary, but we cannot express it with a single line. Attempting to classify this dataset with a line fails, even with a soft margin. The idea for a nonlinear decision boundary is to transform our data somehow, possibly by adding a new dimension, so that it becomes linearly separable. We then try to again maximise the margin. The transformation that is defined, however, is not given explicitly - instead, we use a **kernel**, which essentially describes how we measure *distance* between two points.

For now, we will use a simple transformation as an example - for all our data points (x_1, x_2), we will define $z = (x + y)^2$, essentially mapping them to how far away they are from the origin $(0, 0)$. Once we do so, we can then separate the two classes based on their z value. Note that this explicit transformation of $z = f(x_1, x_2)$ is not how it is typically done.

The linear kernel (with no transformations) clearly performs badly on this dataset, but applying this simple transformation allows us to separate them.

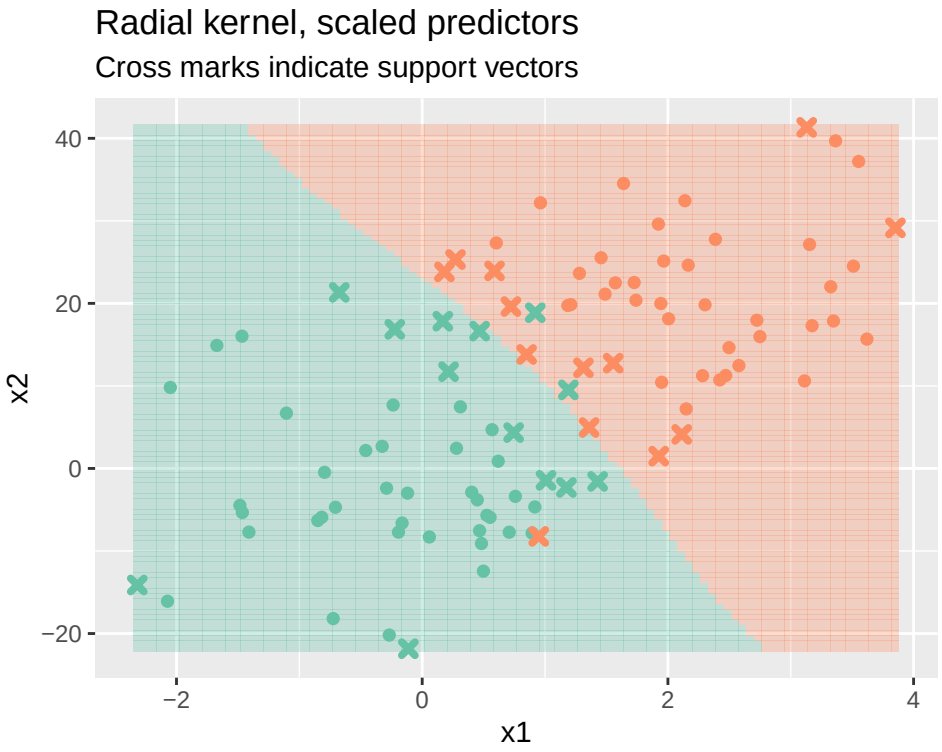


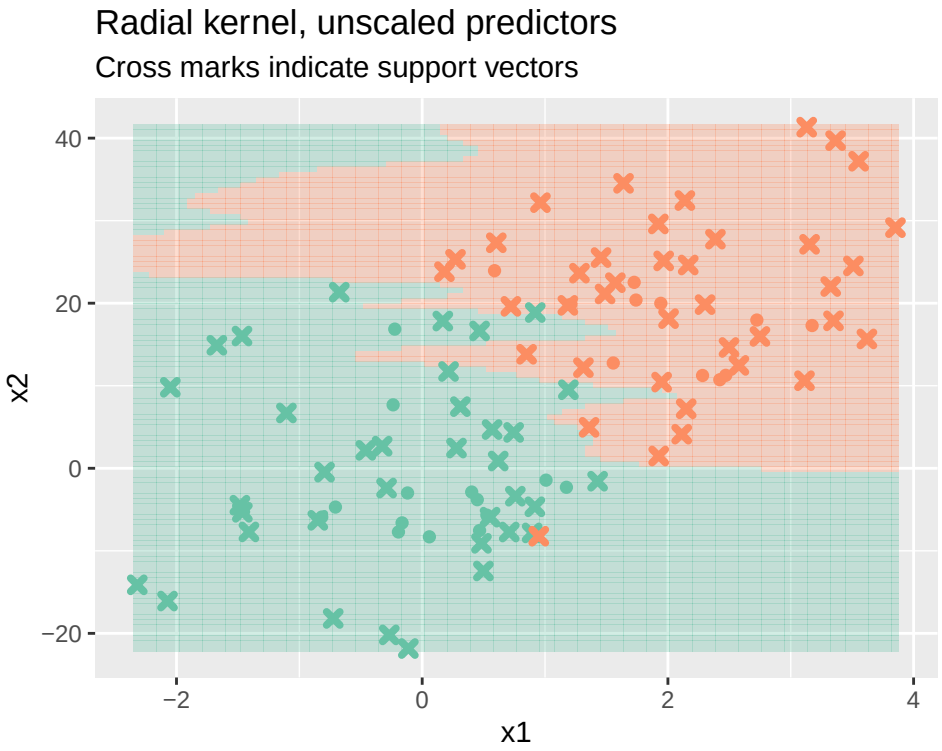


5 Effects of scaling

The scale of each predictor plays a role in SVM, because we are trying to *maximise* a margin, which is the *distance between support vectors*. A predictor with a large scale (standard deviation) can thus distort this margin computation.

Here, we have two predictors, one with a standard deviation of 1, and another with a standard deviation of 10, trained with a sigmoid and radial kernel. You can see that the SVM with unscaled predictors has a much more 'bumpy' decision boundary, and biased its fitting to the predictor with a higher standard deviation. The effects of scaling are not as pronounced with a linear kernel.





6 Revisiting `BreastCancer`

For the third time, we will look at the `BreastCancer` dataset in `mlbench`, and see how support vector machines compare to logistic regression and kNN classification. We will continue to use the same predictors, and pre-process the data in the same way.

```
data("BreastCancer", package="mlbench")
bc <- na.omit(BreastCancer)

bc[2:4] <- sapply(bc[2:4], as.numeric)
bc <- bc %>% mutate(y = factor(ifelse(Class=="malignant", 1, 0))) %>%
  select(Cl.thickness:Cell.shape, y)
str(bc)
```

```
## 'data.frame':    683 obs. of  4 variables:
##  $ Cl.thickness: num  5 5 3 6 4 8 1 2 2 4 ...
##  $ Cell.size   : num  1 4 1 8 1 10 1 1 1 2 ...
##  $ Cell.shape  : num  1 4 1 8 1 10 1 2 1 1 ...
##  $ y           : Factor w/ 2 levels "0","1": 1 1 1 1 1 2 1 1 1 1 ...
##  - attr(*, "na.action")= 'omit' Named int [1:16] 24 41 140 146 159 165 236 25
##   ..- attr(*, "names")= chr [1:16] "24" "41" "140" "146" ...
```

7 Training the model (linear kernel)

Again, we split the data - importantly, with the exact same data as the logistic and kNN models.

```
set.seed(100)
train.idx <- sample(nrow(bc), nrow(bc)*0.8)
train.data <- bc[train.idx,]
test.data <- bc[-train.idx,]
```

We will use the `svm()` function from the `e1071` package to train our SVM models. Its inputs are extremely similar, the only additional options we need to input are the **kernel** to use, and the **parameters** we want it to use, which we will look at later.

```
library(e1071)

m.svm.lin <- svm(y ~ ., train.data, kernel="linear")
summary(m.svm.lin)
```

```
##
## Call:
```

```
## svm(formula = y ~ ., data = train.data, kernel = "linear")
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  linear
##        cost:  1
##
## Number of Support Vectors:  67
##
##  ( 33 34 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

Since the data is 3-dimensional, and we are using a simple linear kernel, the SVM and its separating hyperplane can be visualised relatively easily.

8 Evaluating the model (linear kernel)

As usual, we can look at the evaluation metrics using the test data, looking at the accuracy and the confusion matrix.

```
y <- test.data$y
yhat.lin <- predict(m.svm.lin, test.data)
mean(yhat.lin == y)
```

```
## [1] 0.9416058
```

```
table(predicted=yhat.lin, observed=y)
```

```
##          observed
## predicted  0  1
##         0 82  4
##         1  4 47
```

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{number of observations}} = \frac{82 + 47}{137} = 94.16\% \text{FPR} = \frac{\text{FP}}{\text{number of observed negative classes}} = \frac{4}{86} =$$

9 Kernels

The `svm()` function is able to use some different kernels, which we can specify with the `kernel` parameter. We just used the linear kernel, which only had one parameter, the *cost of misclassification*. We will see how the radial kernel, which has two parameters, performs on our classification task.

The possible values that the `kernel` parameter can take, and each kernel's parameters are: - `linear`, with parameter `cost`; - `radial`, with parameters `cost`, and `gamma`; - `sigmoid`, with parameters `cost`, `gamma`, and `coef0`; - `polynomial` with parameters `cost`, `gamma`, `coef0`, and `degree`.

9.1 Radial kernel

We will use the `tune.svm()` function from `e1071` to tune the parameters for each model. The inputs are similar to the `svm()` function, but we specify all values we wish to test. For a radial basis, we pass in the `cost` and `gamma` values we want to test. Typically, we will space them exponentially apart.

The tuning is via through a cross-validation with 10 folds, as you can see from the output of `summary()`. The best parameters are returned, as well as the `error` and `dispersion` for every parameter set tested. The `error` is the average of `1 - accuracy` across the 10-fold cross-validation, and the `dispersion` is the standard deviation across the 10-fold cross-validation.

We can optionally visualise the error rate across parameters using `plot()`, though I find it difficult to read. Note that the "best performance" shown is the **lowest error rate**, and not the accuracy.

```
set.seed(101)
svm.tune <- tune.svm(y ~ ., data = train.data, kernel = "radial",
                     cost = 10^(-1:2), gamma = c(.1, .5, 1, 2))
summary(svm.tune)
# plot(svm.tune)    # plots error rate
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  gamma cost
##      2  0.1
##
## - best performance: 0.04218855
##
## - Detailed performance results:
##    gamma  cost      error dispersion
## 1    0.1   0.1 0.04771044 0.03143300
## 2    0.5   0.1 0.04404040 0.03262223
```

```
## 3    1.0   0.1 0.04585859 0.03483065
## 4    2.0   0.1 0.04218855 0.03461271
## 5    0.1   1.0 0.04404040 0.03580293
## 6    0.5   1.0 0.04767677 0.03368096
## 7    1.0   1.0 0.04404040 0.03584125
## 8    2.0   1.0 0.04946128 0.03452613
## 9    0.1  10.0 0.04404040 0.03480135
## 10   0.5  10.0 0.04582492 0.03781699
## 11   1.0  10.0 0.05858586 0.03520518
## 12   2.0  10.0 0.07326599 0.02857976
## 13   0.1 100.0 0.04400673 0.03874803
## 14   0.5 100.0 0.06232323 0.04581969
## 15   1.0 100.0 0.07690236 0.03530958
## 16   2.0 100.0 0.06228956 0.02884290
```

The best model is stored in the tune result object, which we can access with `$best.model`. The results here differ from the one in the tutorial due to the different seed used.

```
m.svm.radial <- svm.tune$best.model
yhat.radial <- predict(m.svm.radial, test.data)
mean(yhat.radial == y)
```

```
## [1] 0.9562044
```

```
table(predicted=yhat.radial, observed=y)
```

```
##           observed
## predicted  0  1
##         0 82  2
##         1  4 49
```

9.2 Extra: tuning the cost parameter

You can use the `tune.svm()` function in the same way to find the 'best' `cost` for a linear kernel, though its effects are much less pronounced compared to other parameters in other kernels.

10 Appendix: parameters of each kernel

10.1 Linear kernel

$k(x, y) = x^T y$

Parameter: cost

A high cost (>1000) usually can be treated as a 'hard margin'.

10.2 Radial kernel (radial basis function)

$k(x, y) = \exp(-\gamma \|x - y\|^2)$

Parameters: cost, gamma

10.3 Sigmoid kernel

$k(x, y) = \tanh(\gamma x^T y + \text{coef0})$

Parameters: cost, gamma, coef0

10.4 Polynomial kernel

$k(x, y) = (\gamma x^T y + \text{coef0})^{\text{degree}}$

Parameters: cost, gamma, coef0, degree